

Modificadores de Acesso e Exceções

Aprofundamento de POO, Tratamento de Erros e Listas

Prof. Pedro A.B. Oliveira

FATEC Santana de Parnaíba

Curso: Análise e Desenvolvimento de Sistemas

Disciplina: Linguagem de Programação III - Desenvolver aplicações em JAVA

Aula 03 : 26 de Fevereiro de 2026

Objetivos da Aula de Hoje

Na aula passada, vimos que o **Encapsulamento** protege nossos dados. Mas como controlar exatamente quem pode ver o quê? E como lidar com erros do utilizador?

Nesta aula, vamos:

- Descobrir que existem **4 níveis** de acesso no Java (e não apenas 3).
- Entender a diferença vital entre `private` e `protected`.
- **Evitar que o programa quebre** usando Tratamento de Exceções (`try-catch`).
- Analisar a **Regra de Negócio** do IRPF Progressivo.
- Aprender a trabalhar com `ArrayList` (Listas Dinâmicas) e **Polimorfismo**.

Os 4 Modificadores de Acesso

No Java, o acesso vai do mais restrito ao mais aberto. São eles:

1. private (Privado)

Acesso restrito **apenas à própria classe**. Nem as classes filhas enxergam. É o nível máximo de proteção.

2. Default / Package-Private (Padrão)

Ocorre quando não digita nada. O acesso é permitido para a própria classe e para todas as classes que moram no **mesmo pacote**.

Os 4 Modificadores de Acesso (Cont.)

3. protected (Protegido)

Permite acesso à própria classe, às classes do mesmo pacote e, o mais importante: às **classes filhas (subclasses)**, mesmo que estejam em pacotes diferentes. (O "segredo de família").

4. public (Público)

Nenhuma restrição. **Qualquer classe**, em qualquer lugar do projeto inteiro, pode aceder.

Tabela de Visibilidade: Resumo Prático

Modificador	Mesma Classe	Mesmo Pacote	Classes Filhas	Todo o Projeto
private	Sim	Não	Não	Não
(padrão)	Sim	Sim	Não	Não
protected	Sim	Sim	Sim	Não
public	Sim	Sim	Sim	Sim

Dica: O `protected` é essencial na herança quando a filha precisa manipular as variáveis do pai diretamente!

Qual escolho?

Para não errar na hora de programar:

- **Atributos (Variáveis):** Quase sempre use `private`. Obrigue o uso de Getters e Setters para alterar os dados.
- **Na Herança:** Use `protected` se estiver a criar a Classe Mãe e quiser que as Filhas usem a variável sem precisar de Getters/Setters.
- **Métodos (Ações):** Use `public` para aqueles métodos que devem ser chamados por outras telas ou classes (ex: `calcularLiquido()`).

E quando as coisas correm mal?

Já reparou que se o utilizador digitar "**mil reais**" no campo de Salário, o nosso programa "quebra" e fecha?

No Java, os erros que ocorrem durante a execução do programa são chamados de **Exceções** (*Exceptions*).

- **Erro de Sintaxe:** O código nem compila (ex: faltou um ponto e vírgula).
- **Exceção:** O código compila, mas algo inesperado acontece enquanto o programa está a rodar (ex: tentar converter texto em número, tentar dividir por zero).

O Bloco try-catch

Para evitar que o programa feche abruptamente, precisamos "capturar" esse erro usando o bloco **Try-Catch** (Tentar - Capturar).

```
try {  
    // 1. Tenta executar o código que PODE dar erro  
    double salario = Double.parseDouble(txtSalario.getText());  
    System.out.println("Deu certo!");  
  
} catch (Exception e) {  
    // 2. Código de "resgate". SO EXECUTA se der erro no try.  
    JOptionPane.showMessageDialog(null, "Digite apenas numeros!");  
}
```

Nota: Se a linha do `parseDouble` falhar, o Java pula imediatamente para o `catch`. A linha do `System.out.println` será ignorada.

Desafio: "Blindando"o nosso Botão Calcular

Vamos proteger o código da tela da nossa Folha de Pagamento:

```
private void btnCalcularActionPerformed(java.awt.event.ActionEvent
    evt) {
    try {
        String nome = txtNome.getText();

        // Se o utilizador digitar letras aqui, o Java pula pro
        CATCH
        double salario = Double.parseDouble(txtSalario.getText());
        double liquido = 0;

        Funcionario f = new Funcionario(nome, salario);
        liquido = f.calcularLiquido();
        lblResultado.setText("Salario Liquido: R$ " + liquido);

    } catch (NumberFormatException e) {
        // Captura especificamente o erro de formato de numero
        JOptionPane.showMessageDialog(null, "Erro: Salario invalido!");
    }
}
```

Garantindo a Execução: O Bloco finally

E se precisarmos que um código execute **sempre**, quer dê erro ou não?
Adicionamos o bloco finally.

```
try {  
    double salario = Double.parseDouble(txtSalario.getText());  
    // ... calculos ...  
  
} catch (NumberFormatException e) {  
    JOptionPane.showMessageDialog(null, "Digite apenas numeros!");  
  
} finally {  
    // ESTE BLOCO EXECUTA SEMPRE!  
    // Ideal para: Fechar conexoes de banco de dados, ficheiros,  
    etc.  
  
    txtSalario.setText("");  
    txtSalario.requestFocus(); // Volta o cursor para o campo  
}
```

Regra de Negócio: IRPF Progressivo

O Imposto de Renda Pessoa Física (IRPF) não cobra um percentual fixo de todo o mundo. Ele usa o conceito de **Tabela Progressiva**.

A Fórmula Oficial:

$$\text{Valor do IR} = (\text{Base de Cálculo} \times \text{Alíquota}) - \text{Parcela a Deduzir}$$

- **Base de Cálculo:** É o Salário Bruto *menos* o INSS (para simplificarmos nosso exemplo).
- **Alíquota (%):** A porcentagem cobrada pelo governo.
- **Parcela a Deduzir:** Um "desconto" fixo para garantir que a pessoa não pague imposto sobre a parte do salário que pertencia às faixas inferiores.

A Tabela Progressiva (Valores Didáticos)

Para os nossos cálculos, usaremos os seguintes valores de referência:

Base de Cálculo (R\$)	Alíquota (%)	Deduzir (R\$)
Até 2.428,80	0% (Isento)	0,00
De 2.428,81 até 2.826,65	7,5%	182,16
De 2.826,66 até 3.751,05	15,0%	394,16
De 3.751,06 até 4.664,68	22,5%	675,49
Acima de 4.664,68	27,5%	908,73

Vamos ver como isso funciona na prática com 5 funcionários diferentes.

Exemplos Práticos: Faixas 1 e 2

1. João (Isento) - Base: R\$ 2.000,00

- **Faixa:** Até R\$ 2.428,80.
- **Cálculo:** Isento.
- **IRPF a pagar:** R\$ 0,00

2. Maria (7,5%) - Base: R\$ 2.600,00

- **Faixa:** R\$ 2.428,81 até R\$ 2.826,65.
- **Matemática:** $(2600 \times 0.075) - 182.16$
- **Passo a Passo:** $195.00 - 182.16$
- **IRPF a pagar:** R\$ 12,84

Exemplos Práticos: Faixas 3 e 4

3. Pedro (15%) - Base: R\$ 3.000,00

- **Faixa:** R\$ 2.826,66 até R\$ 3.751,05.
- **Matemática:** $(3000 \times 0.15) - 394.16$
- **Passo a Passo:** 450.00 – 394.16
- **IRPF a pagar:** R\$ 55,84

4. Ana (22,5%) - Base: R\$ 4.000,00

- **Faixa:** R\$ 3.751,06 até R\$ 4.664,68.
- **Matemática:** $(4000 \times 0.225) - 675.49$
- **Passo a Passo:** 900.00 – 675.49
- **IRPF a pagar:** R\$ 224,51

Exemplos Práticos: Faixa 5

5. Carlos (27,5%) - Base: R\$ 6.000,00

- **Faixa:** Acima de R\$ 4.664,68.
- **Matemática:** $(6000 \times 0.275) - 908.73$
- **Passo a Passo:** $1650.00 - 908.73$
- **IRPF a pagar:** R\$ 741,27

O Problema de Programação: Se programarmos isso com `if / else` fixos, toda a vez que o governo alterar os valores da tabela, teremos que recompilar e reinstalar o sistema inteiro nos clientes. Como resolver isso usando Orientação a Objetos?

A Solução: Transformar a Regra num Objeto

A solução é transformar a "Faixa de Imposto" num **Objeto**. O sistema terá um *Cadastro de Faixas* flexível.

```
package model;

public class FaixaIRPF {
    private double limiteMinimo;
    private double limiteMaximo;
    private double aliquota;           // Ex: 0.075 para 7,5%
    private double parcelaDeducacao;  // Ex: 182.16

    // Construtor
    public FaixaIRPF(double min, double max, double aliq, double
        deducacao){
        this.limiteMinimo = min;
        this.limiteMaximo = max;
        this.aliquota = aliq;
        this.parcelaDeducacao = deducacao;
    }
    // ... Getters e Setters ...
}
```

Como guardar várias faixas?

Nós aprendemos em lógica de programação a usar **Vetores (Arrays)** clássicos:

```
// Cria um vetor com tamanho FIXO de 5 posicoes
FaixaIRPF[] tabela = new FaixaIRPF[5];

tabela[0] = new FaixaIRPF(0, 2428.80, 0, 0); // Isento
```

O Problema: O tamanho do Array é *imutável*.

- E se o governo criar uma 6ª faixa (imposto para grandes fortunas)? O programa dará erro (`IndexOutOfBoundsException`).
- E se reduzirem para 3 faixas? Sobrará espaço inútil na memória.

Precisamos de uma estrutura de dados que **cresça e diminua sozinha!**

A Solução Dinâmica: ArrayList

No Java, a melhor ferramenta para isso é a classe `ArrayList` (da biblioteca `java.util`).

Vantagens do ArrayList:

- **Tamanho Dinâmico:** Cresce automaticamente quando se adicionam itens e encolhe quando se removem.
- **Generics (< >):** Avisamos o Java do tipo exato de objeto que vai entrar na lista (ex: `<FaixaIRPF>`), garantindo total segurança.
- **Ferramentas Integradas:** Possui métodos nativos prontos para procurar, remover, substituir e contar itens.

Sintaxe Básica do ArrayList

```
import java.util.ArrayList;

// 1. Criando a lista vazia (Generics: avisamos que guarda Faixas)
ArrayList<FaixaIRPF> listaFaixas = new ArrayList<>();

// 2. Adicionando itens (.add)
listaFaixas.add(new FaixaIRPF(0, 2428.80, 0, 0));
listaFaixas.add(new FaixaIRPF(2428.81, 2826.65, 0.075, 182.16));

// 3. Obtendo o tamanho da lista (.size)
int quantidade = listaFaixas.size(); // Retornara 2

// 4. Lendo um item especifico (.get)
// Pega o item do indice 1 (que e a segunda faixa inserida)
FaixaIRPF faixa = listaFaixas.get(1);
```

Subindo o Nível: Polimorfismo em ArrayList

Até agora fizemos uma lista de uma classe só (FaixaIRPF).

Mas imagine processar uma Folha de Pagamento real com 5.000 empregados. Teríamos que criar uma `ArrayList<Funcionario>` comum, outra `ArrayList<Gerente>` e outra `ArrayList<Diretor>`?

NÃO! É aqui que entra o grande poder do **Polimorfismo de Inclusão**.

Como o Gerente "*é um*" Funcionário (relação de Herança), o Java permite colocar **todos os tipos de funcionários (Filhos) dentro de uma única Lista Mãe**.

O Polimorfismo na Prática

```
// Criamos UMA UNICA lista do tipo da Classe PAI (Funcionario)
ArrayList<Funcionario> folhaPagamento = new ArrayList<>();

// O Polimorfismo permite adicionar objetos de classes FILHAS diferentes!
folhaPagamento.add(new Funcionario("Joao", 2000));
folhaPagamento.add(new Gerente("Maria", 5000, 1000));

// Percorrendo a lista inteira (Usando a sintaxe "For-Each")
for (Funcionario f : folhaPagamento) {

    // O Java descobre sozinho em tempo de execucao se devera
    // usar a formula base ou a formula com bonus do gerente!
    double liquido = f.calcularLiquido();

    System.out.println(f.getNome() + " recebe: R$ " + liquido);
}
```